

Architecture & Trade-offs

This document describes decisions and production considerations derived from the current in-memory ledger implementation. It intentionally distinguishes **what the code implements today** from **production behaviors not implemented yet**.

1) Append-only at scale

What the current code does

The replay engine is append-only and computes balances by scanning postings with:

- `close_for_day()` in `ledger_core/core.py`.

That function is called repeatedly by:

- overdraft fee reconciliation,
- daily interest accrual,
- daily reporting,
- replay checkpoints.

What breaks first at 100× volume

At higher volume, CPU/read amplification is likely to degrade first, before raw append throughput.

Why:

- each close computation scans a growing postings list,
- and close computations are repeated many times across replay phases.

This creates repeated-scan behavior that trends toward approximately quadratic replay work as event count grows.

Unbounded state growth

For production-scale horizons, these structures grow without bound:

- postings
- `seen_event_ids`
- authorization records/history
- errors
- checkpoints
- production-equivalent fee/history indexes

Note: `fee_days` is bounded in this six-day exercise but would scale with time horizon in a generalized engine.

Cheapest structural change (before platform rebuild)

Lowest-cost improvement while preserving append-only truth:

1. keep immutable journal semantics unchanged,
2. add derived per-account/day balance deltas (or a compact close index),
3. make checkpoints optional outside test/debug mode.

In this fixed six-day model, a tiny per-account six-day derived structure avoids scanning all postings for every close query.

Next production step

After the low-cost index/snapshot step:

- move immutable journal to durable partitioned storage,
- build rebuildable account/day materialized projections.

Important invariant:

snapshot/projection is not the source of truth; immutable history remains authoritative.

2) Value-dated entries in production

Operational surface

Maintaining both booking time and value date creates a two-time-axis system. Backvalued entries can retroactively change historical effective closes. Operationally this impacts:

- fee and interest recalculation,
- already-issued statements,

- prior customer notifications,
- reconciliation windows and GL interfaces,
- risk/financial-crime downstream views,
- customer dispute handling,
- cross-system consistency when services observe history at different times.

UAE regulatory/control surface (primary sources)

These sources do not prescribe this specific algorithm, but they create engineering control obligations around process integrity, records, and customer impact handling:

1. ****CBUAE Operational Risk and Operational Resilience Regulation C 1/2026 — Article (7) Internal Control System****

URL: <https://rulebook.centralbank.ae/en/rulebook/article-7-internal-control-system>

Observed status/effective metadata at review time (2026-09-18): *In-Force*, effective from *2026-09-14*.

2. ****CBUAE Consumer Protection Standards****

URL: <https://rulebook.centralbank.ae/en/rulebook/consumer-protection-standards>

Observed status metadata at review time (2026-09-18): *In-Force* (retail-consumer context; applicability depends on product/customer type). Effective date is not explicitly shown on this standards page metadata.

3. ****CBUAE AML/CFT Rulebook — Record Keeping****

URL: <https://rulebook.centralbank.ae/en/rulebook/4-record-keeping>

Observed in related rulebook record-keeping section metadata at review time (2026-09-18): *In-Force*. Effective date varies by instrument/section; one indexed record-keeping section (4.4.6) shows *2025-11-07*.

Engineering interpretation boundary:

These sources create a control surface around traceability, approvals, segregation/monitoring, and long-horizon record reconstruction. They are not read here as instructions for a single ledger data structure.

One concrete pre-go-live control: Backvalue Impact Gate

For any posting where `value_date < booking_date`, require pre-append impact analysis and governance.

Minimum gate outputs:

- affected day closes,
- fee delta,
- interest delta,
- statement/notification correction impacts,
- reconciliation period impacts.

Minimum gate controls:

- reason code,
- source/provenance metadata,
- immutable audit envelope,
- maker-checker approval when customer-visible amounts change or materiality threshold is crossed.

After append, downstream consumers should receive correction signals/events rather than silently drifting to a new history.

3) Authorization lifecycle

Implemented model (exact)

In current code, authorization statuses are:

- ACTIVE
- DECLINED
- SETTLED

Other than a matching settlement, the implemented model has exactly one terminal outcome:

- DECLINED at authorization time.

Meaning and mandated behavior in this model:

- no hold created,
- deterministic decline reason,
- decision preserved for audit,
- later backdated events do not retroactively flip decline to approval.

Clarifications:

- ACTIVE can remain active for the whole window if never settled,
- settlement referencing unknown/inactive auth is an error outcome, not a lifecycle transition.

Production lifecycle states intentionally not implemented (gaps)

These are ****production gaps****, not current behavior:

- EXPIRED
 - Scenario: merchant never captures within hold validity window.
 - Production behavior: auto-release hold with expiry event and customer-available-balance update.
- VOIDED / AUTHORIZATION_REVERSED
 - Scenario: merchant/network cancel after authorization.
 - Production behavior: release hold immediately; append explicit reversal/void event.
- PARTIALLY_CAPTURED + remainder released
 - Scenario: settlement lower than held amount.
 - Production behavior: settle captured amount, release remainder, persist full chain.
- ADMIN_RELEASED / NETWORK_RELEASED
 - Scenario: operational or network intervention.
 - Production behavior: privileged workflow with dual control, reason capture, and audit trail.

4) What we cut and why

This implementation is intentionally narrow; each simplification defers concrete production risk.

Simplifications and Deferred Risks (compact mapping)

Simplification	Deferred production risk
In-memory single process	Crash recovery, HA, and cross-node consistency exposure
No durable event store/snapshots	Replay latency growth and retention/forensics pressure
Linear close scans	CPU/read amplification under higher event volume
Minimal auth lifecycle states	Missing expiry/void/partial-capture handling
No downstream correction adapters	GL drift and customer-notification mismatch risk